Documento de Arquitectura de Software (DAS)

Caso 7 — Alerta temprana de deslizamientos

Curso: Arquitectura de Software — Uniremington
Docente: Gloria Amparo Lora Patiño
Estudiante: Alejandro Acosta
Versión del documento: 1.0 — Fecha: 07/11/2026
Repositorio: alerta-temprana-deslizamientos

0. Enunciado del Caso 7 (texto original)

Contexto del caso

Una red de 8 sensores de humedad instalados en zona de ladera reporta datos por lotes cada hora, vía SMS, a un celular que un técnico revisa manualmente. No hay presupuesto para reemplazar los sensores.

Palabras del cliente

“El deslizamiento de la vereda El Roble ocurrió a las 3 de la madrugada. El reporte del sensor llegó a las 4am, cuando el técnico finalmente lo revisó. Para entonces ya era tarde.”

Datos del caso

Infraestructura fija: 8 sensores existentes, no reemplazables por restricción de presupuesto.

Frecuencia actual: reporte por lotes cada hora, revisión manual.

Atributo de calidad crítico: disponibilidad y tiempo de respuesta (una alerta tardía no sirve).

Restricción técnica dura: no se puede cambiar el hardware de los sensores, solo el procesamiento de los datos que ya envían.

Reto para el arquitecto

Diseñar cómo procesar y alertar sobre los datos que ya emiten los sensores existentes de forma casi inmediata, sin poder modificar el hardware, priorizando disponibilidad y tiempo de respuesta por encima de cualquier otro atributo.

Ficha de Drivers Arquitectónicos — Entregable 1

Caso 7 · Alerta temprana de deslizamientos · Cierre Unidad 1

1. Requisitos funcionales (RF-01 a RF-10)

ID	Requisito funcional	Origen / justificación
RF-01	Recibir y registrar los SMS entrantes de los 8 sensores	Explícito: “reporta datos por lotes cada hora, vía SMS”
RF-02	Interpretar el contenido de cada SMS (sensor, hora, valor de humedad)	Derivado: necesario para procesar sin tocar el hardware
RF-03	Almacenar el histórico de lecturas por sensor	Derivado: se necesita para detectar tendencia, no solo el dato actual
RF-04	Evaluar cada lectura contra un umbral de riesgo configurado	Derivado del reto: “alertar sobre los datos que ya emiten”
RF-05	Detectar tendencia de aumento sostenido en lecturas consecutivas	Derivado de la cita del cliente: la alerta a tiempo exigía ver el patrón antes del lote crítico
RF-06	Generar una alerta cuando se supere el umbral o se detecte tendencia de riesgo	Explícito: “alertar... de forma casi inmediata”
RF-07	Notificar la alerta al responsable por un canal más inmediato que la revisión manual del celular	Derivado: la revisión manual fue la causa raíz del retraso en el caso
RF-08	Permitir configurar/ajustar los umbrales de riesgo	Derivado: sin esto no hay forma de calibrar el sistema al escenario real
RF-09	Registrar historial de alertas generadas (trazabilidad)	Derivado de buenas prácticas de trazabilidad
RF-10	Consultar el estado de cada sensor (última lectura, si reporta con normalidad)	Derivado: mitiga el riesgo de sensor caído o dañado

2. Atributos de calidad — ISO/IEC 25010

AC	Característica	¿Aplica?	Justificación	Prioridad
AC-01	Adecuación funcional	Sí	Debe procesar correctamente las lecturas de los 8 sensores y generar la alerta correspondiente sin omitir eventos de riesgo real	Media
AC-02	Eficiencia de desempeño	Sí	“Alertar... de forma casi inmediata”; el caso mide el fracaso en horas (3am vs 4am)	Alta
AC-03	Compatibilidad	Sí	Debe integrarse con el canal SMS y los sensores ya existentes, sin poder cambiarlos	Media
AC-04	Usabilidad	Sí	El técnico revisa manualmente hoy; el nuevo sistema debe operarlo alguien con conocimiento técnico bajo/medio	Media
AC-05	Confiabilidad (fiabilidad)	Sí	“Atributo de calidad crítico: disponibilidad”; un solo técnico y un solo celular es punto único de falla	Alta
AC-06	Seguridad	Sí	No hay dato explícito, pero una alerta falsa o manipulada sobre riesgo de vida tiene consecuencia grave	Baja
AC-07	Mantenibilidad	Sí	Restricción de hardware fijo: toda evolución (umbrales, lógica de tendencia) debe hacerse solo en software	Alta
AC-08	Portabilidad	No aplica	El caso no menciona necesidad de correr en otra plataforma o migrar de entorno	No aplica

3. Restricciones técnicas

RT	Restricción	Origen
RT-01	No se puede reemplazar ni modificar el hardware de los 8 sensores; solo se interviene el procesamiento posterior	Explícito en el caso
RT-02	El canal es SMS por lotes cada hora; no hay flujo de datos continuo ni streaming	Explícito: “reporta datos por lotes cada hora, vía SMS”
RT-03	El único punto de recepción actual es un celular revisado manualmente; no hay infraestructura de recepción adicional	Derivado del caso
RT-04	Solo se dispone de humedad como variable; no hay lluvia, inclinación u otros sensores	Derivado, limita cualquier modelo predictivo

4. Restricciones administrativas

RA	Restricción	Origen
RA-01	No hay presupuesto para reemplazar los sensores	Explícito en el caso
RA-02	No hay evidencia de presupuesto para infraestructura adicional (servidores, nube, personal técnico)	Derivado, coherente con restricciones como problema de inversión
RA-03	No hay turno de guardia nocturno confirmado; el reporte de las 3am solo se revisó hasta las 4am	Derivado de la cita del cliente

5. Pregunta al cliente

¿Los reportes de las horas previas al deslizamiento (por ejemplo 1am y 2am) ya mostraban un aumento progresivo de humedad, o el salto ocurrió únicamente en el lote de las 3am?

Esta pregunta expone el supuesto que sostiene todo el diseño: si hubo tendencia previa, el sistema necesita detectar tendencia (RF-05) y no solo un umbral fijo; si fue un salto súbito, ni la alerta más rápida habría llegado a tiempo, y el problema real es el intervalo físico de una hora entre lecturas, no el software.

Sección 1: Introducción y Drivers Arquitectónicos

1.1 Visión general del negocio y alcance

Problema de negocio: una red de 8 sensores de humedad instalados en zona de ladera reporta datos por lotes cada hora, vía SMS, a un celular revisado manualmente por un técnico. La revisión tardía impide alertar a tiempo, como ocurrió en el deslizamiento de la vereda El Roble.

Propuesta de solución: el sistema permite al equipo de vigilancia procesar automáticamente los reportes SMS de los sensores existentes para generar y notificar alertas de riesgo de deslizamiento sin depender de la revisión manual de un único técnico.

Alcance (in-scope): recepción e interpretación de SMS de los 8 sensores existentes; almacenamiento histórico de lecturas; evaluación de umbral y detección de tendencia; generación y notificación de alertas por múltiples canales; configuración de umbrales; consulta de estado de sensores e historial de alertas mediante una interfaz web mínima.

Estado del alcance en la versión 1.0: lo implementado es la base de datos de cinco tablas (5.2) y el CRUD de la entidad Alerta con autenticación JWT y control por rol (5.3). La recepción de SMS por webhook, la evaluación de umbral y de tendencia, la notificación por correo y Telegram, la configuración de umbrales y la interfaz web están diseñadas en las Secciones 2 a 4, pero aún no están implementadas: se declaran como «Pendiente» en la Vista de Despliegue (3.5) y su efecto sobre los atributos de calidad se evalúa en la Sección 6.

Fuera de alcance (out-of-scope): reemplazo o modificación del hardware de los sensores; predicción climática con variables distintas a humedad (lluvia, sismos); coordinación de evacuación o respuesta de emergencia física; aplicación móvil nativa para el ciudadano final; cambio del canal de transmisión de los sensores (sigue siendo SMS).

1.2 Historias de Usuario arquitectónicamente significativas

ID	Historia de usuario	¿Por qué es significativa?
HU-01	Como sistema, quiero generar y registrar una alerta cuando una lectura supere el umbral o muestre tendencia de riesgo, para notificar el peligro con el menor retraso posible.	Ejercita persistencia, lógica de negocio crítica e integración con canal de notificación. Base del CRUD del Corte Vertical (entidad “Alerta”).
HU-02	Como administrador, quiero configurar los umbrales de riesgo por sensor, para ajustar la sensibilidad del sistema sin depender de cambios en el hardware.	Ejercita seguridad/autorización y conecta directo con la restricción técnica de hardware fijo.
HU-03	Como técnico, quiero consultar el estado y última lectura de cada sensor, para detectar si alguno dejó de reportar.	Ejercita confiabilidad y tolerancia a fallos.

1.3 Atributos de calidad prioritarios — escenarios medibles

Atributo	Escenario de calidad	Métrica objetivo	¿Cómo se verificará?
Eficiencia de desempeño	Llega un SMS con una lectura de humedad, en operación normal	El sistema procesa y evalúa la lectura en ≤ 5 segundos desde su recepción	Comparar timestamp de llegada del SMS vs. timestamp de evaluación, en logs
Confiabilidad	Se genera una alerta de riesgo, en cualquier horario incluida la madrugada sin guardia activa	La alerta se entrega por al menos 2 canales/destinatarios independientes en el 100% de los casos	Prueba funcional: desactivar un canal y confirmar que el otro entrega la alerta
Mantenibilidad	Se necesita ajustar el umbral de riesgo o la regla de tendencia	El cambio afecta máximo 1-2 archivos de configuración, sin tocar el módulo de recepción de SMS ni el esquema de BD	Revisión de código del corte vertical

Nota: el tiempo de procesamiento (≤ 5 s) mide la latencia del software, no el intervalo de una hora entre lecturas del sensor, el cual es una restricción de hardware (RT-02) que no puede modificarse.

1.4 Restricciones

Tipo	Restricción
Técnica	No se puede reemplazar ni modificar el hardware de los sensores
Técnica	Canal SMS por lotes cada hora, sin streaming continuo
Operativa	Sin presupuesto para sensores ni infraestructura adicional
Operativa	Sin evidencia de turno de guardia nocturno
De tiempo	Un solo desarrollador (proyecto individual), con plazo corto y tecnología de dominio previo (Java con Spring Boot y React). Corte vertical funcional Clase 6 (07-nov); DAS v1.0 Clase 7 (14-nov); entrega final Clase 8 (21-nov). Consecuencia: se implementa el corte vertical (5.3) y el resto queda pendiente (1.1).
Normativa	No identificada explícitamente en el caso

Sección 2: Estrategia Arquitectónica y Tecnológica

2.1 Patrón arquitectónico principal

Patrón seleccionado: N-Capas (presentación, negocio y datos).

Justificación: Se selecciona el patrón N-Capas (presentación, negocio y datos) porque el atributo prioritario Mantenibilidad (AC-07) exige que ajustar un umbral o la regla de tendencia afecte máximo 1-2 archivos de configuración, sin tocar el módulo de recepción de SMS ni el esquema de la base de datos; la separación en capas concentra las reglas en una sola capa de negocio, aislada de la entrada y de la persistencia. Esa misma capa expone los canales de notificación detrás de una interfaz común con al menos dos implementaciones independientes, lo que sostiene la Confiabilidad (AC-05): si un canal falla, el otro entrega la alerta. Al desplegarse el backend como una sola aplicación, la lectura se procesa dentro del mismo proceso, sin saltos de red internos, lo que favorece la Eficiencia de desempeño (AC-02, ≤ 5 s). Se descarta la Arquitectura Hexagonal porque su estructura adicional contradice la restricción de tiempo (sección 1.4) de un proyecto individual con un solo desarrollador, y se descartan Microservicios y la arquitectura orientada a eventos con broker porque su costo operativo contradice la restricción de presupuesto (RA-02) y añade puntos de falla frente a la Confiabilidad (AC-05).

2.2 Pila tecnológica

Cada tecnología se conecta con un atributo de calidad prioritario (sección 1.3) o con una restricción (sección 1.4), y se indica la alternativa descartada.

Capa	Tecnología elegida	Versión	Alternativa descartada	Justificación
Lenguaje backend	Java	21 LTS	Python	Restricción de tiempo (sección 1.4): lenguaje de dominio previo del estudiante (dos asignaturas cursadas con Spring Boot) en un proyecto individual con plazo corto; Python implicaría una curva de aprendizaje nueva.
Framework	Spring Boot	4.1.x	FastAPI, Django	AC-07 (Mantenibilidad): su estructura Controller-Service-Repository materializa las capas del patrón N-Capas y aísla las reglas de negocio en la capa de servicio; incluye validación de datos y documentación OpenAPI. Restricción de tiempo: dominio previo del estudiante.
Acceso a datos	Spring Data JPA (Hibernate) + Flyway	Según Spring Boot	JDBC / SQL nativo	AC-07: mantiene el modelo de datos separado de las reglas de negocio mediante repositorios y versiona los cambios del esquema con migraciones.

Capa	Tecnología elegida	Versión	Alternativa descartada	Justificación
Base de datos	PostgreSQL	18	MongoDB	Datos relacionales (sensor, lectura, alerta) que requieren integridad referencial y restricciones (FK, CHECK, UNIQUE) para el historial de RF-03 y RF-05; licencia libre (RA-02).
Frontend	React (Vite) como aplicación separada	React 19.x	HTML y JavaScript servidos desde Spring Boot	AC-04 (Usabilidad): interfaz mínima para consultar alertas y estado de sensores (HU-03) y configurar umbrales (HU-02). Restricción de tiempo: dominio previo del estudiante en React; consume la API REST del backend con JWT.
Autenticación	Spring Security + JWT, con BCrypt y roles administrador y técnico	Según Spring Boot	Sesiones en servidor	AC-06 (Seguridad) y HU-02: solo el administrador modifica umbrales, porque un umbral alterado puede suprimir una alerta real. JWT evita mantener estado de sesión en el servidor.
Pruebas	JUnit + Postman	Según Spring Boot (JUnit)	Pruebas manuales	AC-02 y AC-05: verificación repetible de la evaluación en ≤ 5 s y de la entrega por 2 canales, según los escenarios de la sección 1.3.
Control de versiones	Git + GitHub	—	Ninguna	Restricción del curso: el DAS, los ADRs y el código se versionan en el repositorio del proyecto.
Entrada de SMS	Endpoint HTTP (webhook) que recibe el SMS; en desarrollo se simula con Postman	—	Contratar un gateway SMS de pago en esta etapa	RT-02 y RA-02: el canal de los sensores sigue siendo SMS; el mensaje llega al sistema como una petición HTTP, sin costo de un gateway de pago.
Canales de salida	Correo (SMTP) + Telegram Bot	—	SMS saliente por gateway de pago	AC-05 (Confiabilidad): dos canales independientes, ambos sin costo (RA-02), de modo que la falla de uno no impide la entrega por el otro.

Nota: las versiones marcadas como "Según Spring Boot" se heredan de su gestión de dependencias; todas las versiones se confirman al crear el proyecto.

Sección 3: Vistas de Arquitectura (Modelo C4 y UML)

Esta sección presenta la arquitectura mediante el Modelo C4 con notación UML estándar. Cada vista incluye su diagrama, una leyenda y un párrafo de lectura. Las fuentes de los diagramas (.puml) y sus exportaciones (.png) están en la carpeta docs/diagramas/ del repositorio.

3.1 Vista de Contexto — C4 Nivel 1

Muestra el sistema como una caja negra: quién lo usa y con qué dispositivos y sistemas externos se comunica, sin mostrar ninguna tecnología interna.

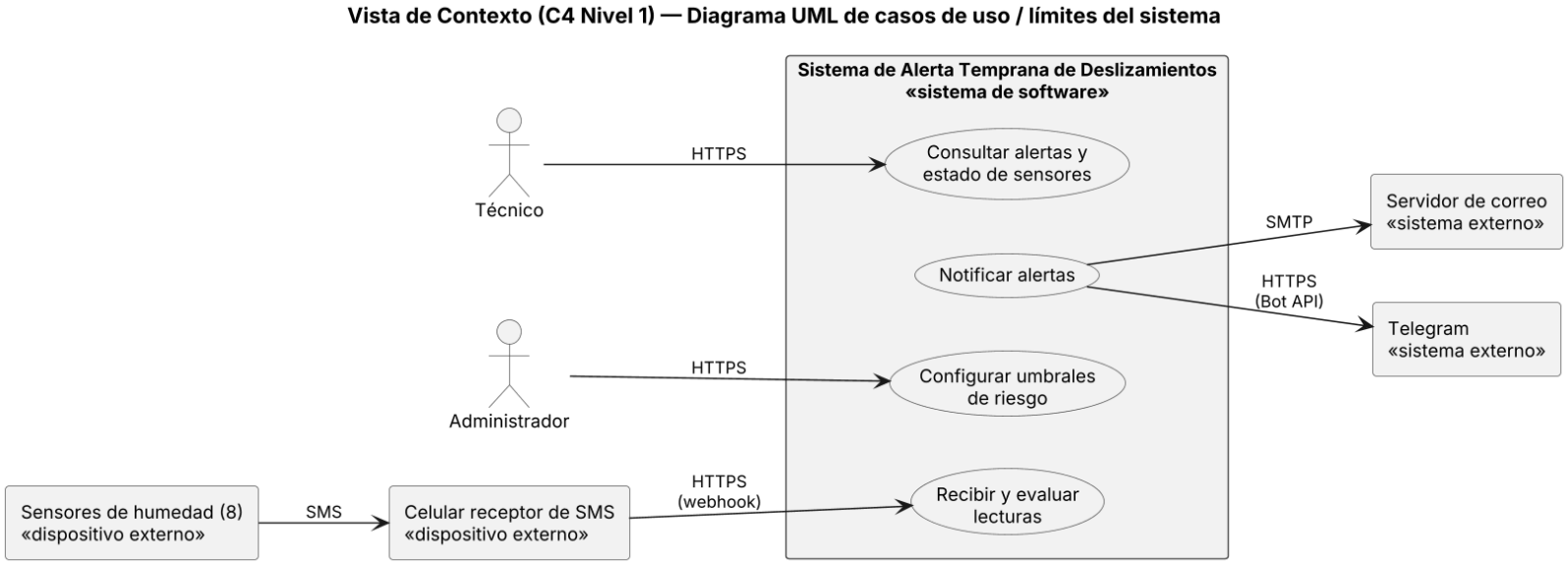


Diagrama UML de la Vista de Contexto (C4 Nivel 1)

Leyenda

Elemento del diagrama	Qué representa
Figura de persona	Actor humano que usa el sistema.
Rectángulo «dispositivo externo»	Equipo físico fuera del sistema que no se modifica (los sensores y el celular receptor de SMS).
Rectángulo «sistema externo»	Software de terceros con el que el sistema se comunica (servidor de correo y Telegram).
Recuadro grande con elipses	Límite del sistema y sus capacidades principales.
Flecha con etiqueta	Apunta hacia quien recibe la comunicación; la etiqueta indica el protocolo.

Actores y relación con las historias de usuario (sección 1.2): el administrador (HU-02) y el técnico (HU-03) son los actores humanos y usan el sistema por HTTPS. La HU-01 la ejecuta el propio sistema al recibir y evaluar las lecturas, por eso no aparece como un actor externo. Cada interfaz externa figura con su protocolo: SMS desde los sensores, HTTPS (webhook) desde el celular receptor, SMTP hacia el servidor de correo y HTTPS (Bot API) hacia Telegram.

Lectura en 30 segundos: ocho sensores de humedad envían sus lecturas por SMS a un celular que ya existe en el caso. Ese celular las reenvía por HTTPS (webhook) al sistema, que las evalúa, genera alertas y las notifica por correo (SMTP) y por Telegram (HTTPS). El técnico consulta las alertas y el estado de los sensores, y el administrador configura los umbrales de riesgo, ambos por HTTPS. Los sensores y el celular no se modifican: todo el valor sale del software.

Nota: el reenvío del SMS desde el celular receptor hacia el webhook es un supuesto de diseño por confirmar con el cliente. El caso establece que los sensores envían sus lecturas por SMS a un celular y que su hardware no puede modificarse (RT-01), pero no indica cómo llega ese mensaje a un sistema de software. El mecanismo de reenvío aún no está definido y deberá ser compatible con la restricción de presupuesto (RA-02), es decir, sin un gateway SMS de pago; mientras tanto, en desarrollo el SMS se simula con Postman (sección 2.2).

3.2 Vista de Contenedores — C4 Nivel 2

Abre la caja negra del sistema: muestra las piezas que se ejecutan por separado y los almacenes de datos, cada una con su tecnología entre corchetes.

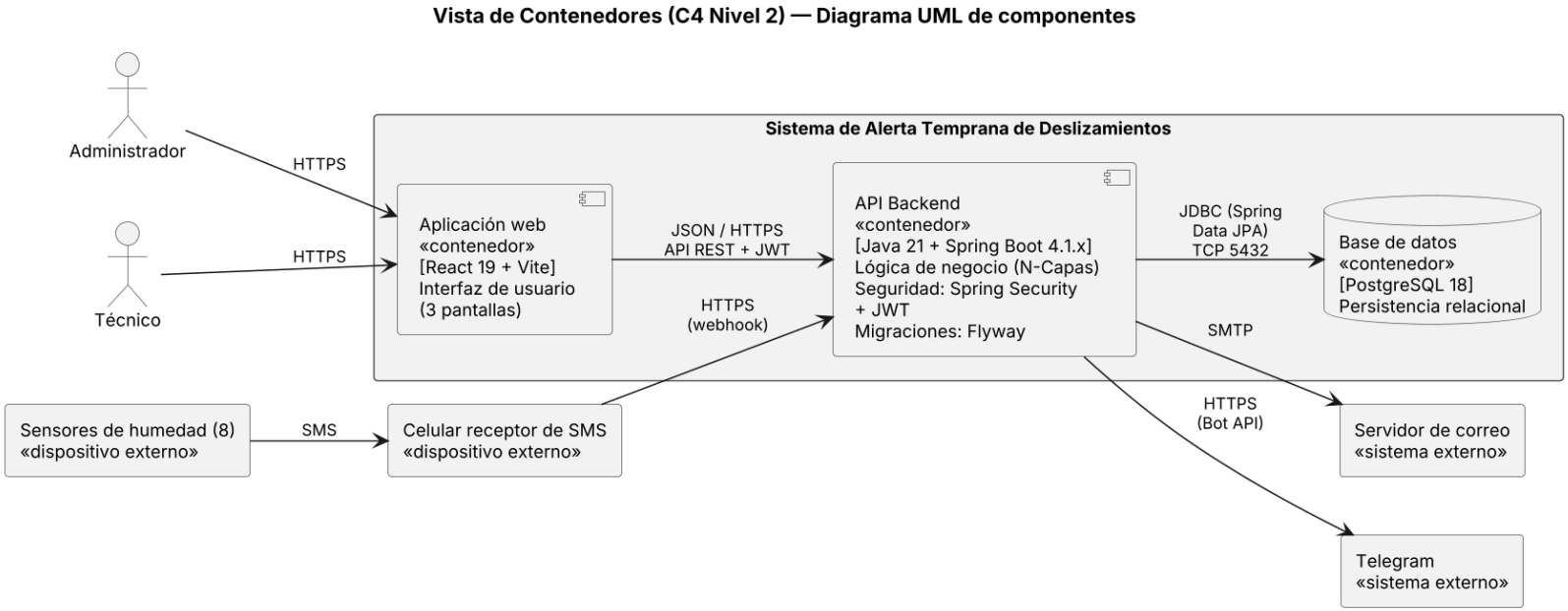


Diagrama UML de componentes de la Vista de Contenedores (C4 Nivel 2)

Leyenda

Elemento del diagrama	Qué representa
Componente «contenedor» con la tecnología entre corchetes	Pieza propia que se ejecuta por separado: la aplicación web y la API.
Cilindro «contenedor»	Almacén de datos propio: la base de datos.
Recuadro grande	Límite del sistema: lo de adentro es propio, lo de afuera no se controla.
Rectángulos «dispositivo externo» y «sistema externo»	Los mismos elementos externos de la Vista de Contexto.
Flecha con etiqueta	Apunta hacia quien recibe la comunicación; la etiqueta indica el protocolo (y el puerto cuando aplica).

Correspondencia con la pila tecnológica (sección 2.2): la aplicación web es React 19 con Vite; la API es Java 21 con Spring Boot 4.1.x, que incluye Spring Security con JWT, Spring Data JPA y Flyway; y la base de datos es PostgreSQL 18. El webhook es la entrada de los SMS, y el correo (SMTP) y Telegram son los canales de salida. Las tres capas del patrón N-Capas (sección 2.1) están dentro del contenedor API Backend.

Lectura en 30 segundos: el técnico y el administrador entran por HTTPS a la aplicación web (React), que habla con la API (Spring Boot) en JSON por HTTPS con JWT. La API es la única pieza que accede a la base de datos PostgreSQL (JDBC, puerto 5432). Además recibe por webhook HTTPS los SMS que reenvía el celular receptor y envía las alertas por correo (SMTP) y por Telegram (HTTPS). Las tres piezas dentro del recuadro son propias; los dispositivos y sistemas de afuera no se controlan.

3.3 Vista de Dominio — Diagrama de Clases UML

Muestra los datos que guarda el sistema: cada caja es una tabla de la base de datos y cada línea, una relación entre tablas. Es el modelo que implementa el ORM (Spring Data JPA).

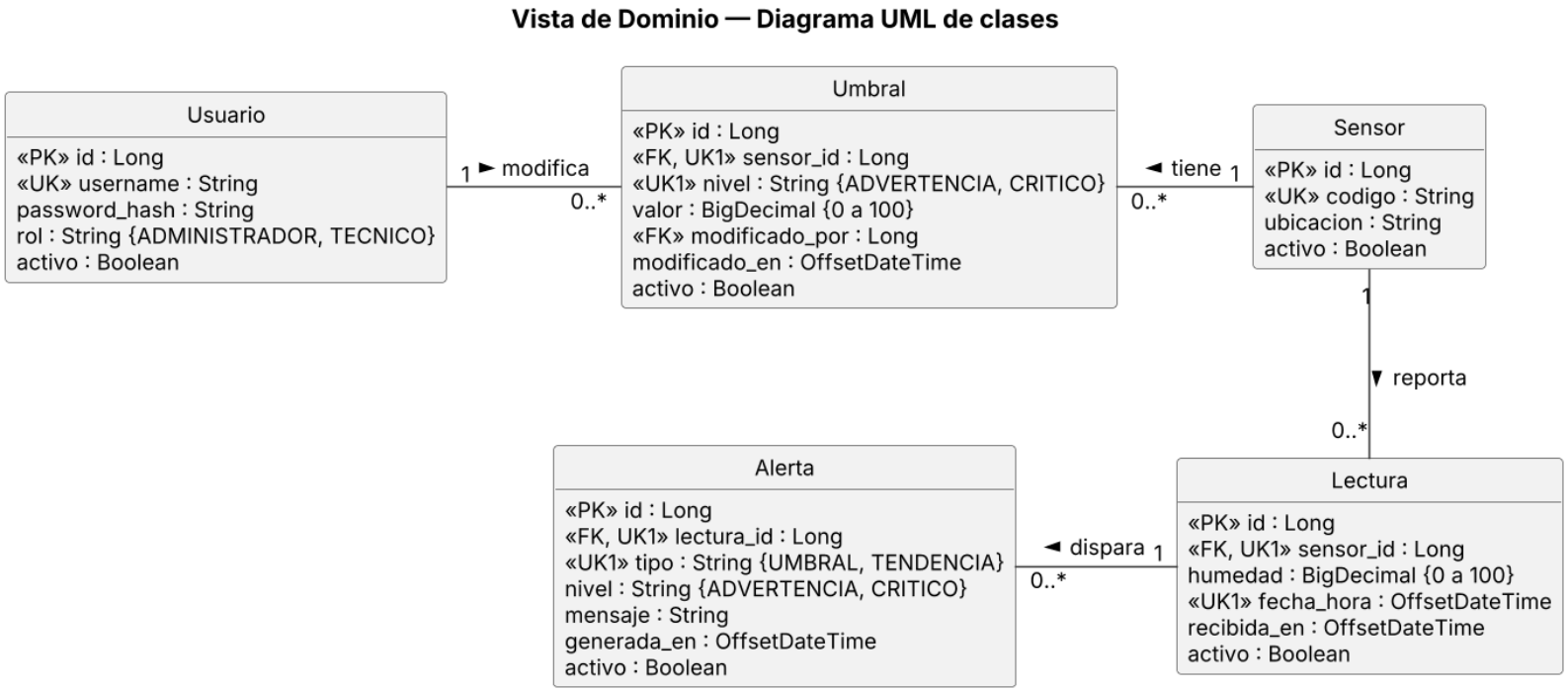


Diagrama UML de clases de la Vista de Dominio

Leyenda

Elemento del diagrama	Qué representa
Caja con el nombre de la clase y sus atributos	Una entidad del sistema, que se guarda como una tabla. Cada línea de la caja es un atributo con su tipo (nombre : Tipo) y se guarda como una columna con el mismo nombre.
«PK» y «FK»	«PK» es la llave primaria: identifica de forma única cada fila. «FK» es una llave foránea: guarda la llave primaria de otra clase y es lo que materializa la relación que une las dos cajas.
«UK» y «UK1»	Atributo con valor único. Los atributos que llevan el mismo número (UK1) forman una sola restricción conjunta: por ejemplo, no puede repetirse la pareja sensor_id y fecha_hora.
Valores entre llaves { }	Valores permitidos del atributo (restricción CHECK), por ejemplo {ADVERTENCIA, CRITICO} o {0 a 100}.
Línea con números en los extremos	Relación entre clases. El número de cada extremo dice cuántas filas de ese lado se relacionan con una del otro lado: 1 es exactamente una; 0..* es ninguna o muchas. El verbo se lee en el sentido que marca el triángulo.
activo : Boolean	Borrado lógico: en lugar de borrar la fila se marca como falso. Todas las clases lo tienen.

Entidades y relación con las historias de usuario (sección 1.2): la HU-01 usa Lectura y Alerta, y consulta Sensor y Umbral para evaluar; la HU-02 usa Umbral y Usuario (quién cambió el límite); la HU-03 usa Sensor y Lectura (la última lectura de cada sensor). Alerta es la entidad del CRUD del corte vertical (sección 5). Umbral es una clase propia para que cambiar un límite sea editar un dato (HU-02) y no tocar el esquema ni la recepción de SMS (AC-07). El rol es un atributo de Usuario porque solo hay dos roles fijos. Alerta apunta a la Lectura que la disparó y no directamente al Sensor, para no guardar dos veces el mismo dato.

Lectura en 30 segundos: el sistema guarda cinco cosas. Un Sensor reporta muchas Lecturas, y de cada Lectura puede salir ninguna o varias Alertas (como máximo una de cada tipo: por umbral y por tendencia). Cada Sensor tiene sus Umbrales de riesgo, y cada Umbral recuerda qué Usuario lo modificó. Las llaves (PK, FK, UK) dicen cómo se identifica y se enlaza cada fila, y la columna activo permite el borrado lógico.

Nota: las clases no tienen métodos porque esta vista modela los datos que implementa el ORM; lo que el sistema hace con ellos se muestra en la Vista Dinámica (3.4). Los tipos del diagrama son de Java; la sección 5.2 muestra su equivalente en PostgreSQL.

3.4 Vista Dinámica — Diagrama de Secuencia UML

Muestra cómo el escenario primario (HU-01) atraviesa las capas del patrón N-Capas, desde que llega un SMS hasta que se guarda y se notifica la alerta. Incluye el camino feliz y dos flujos alternos: 401 y 409.

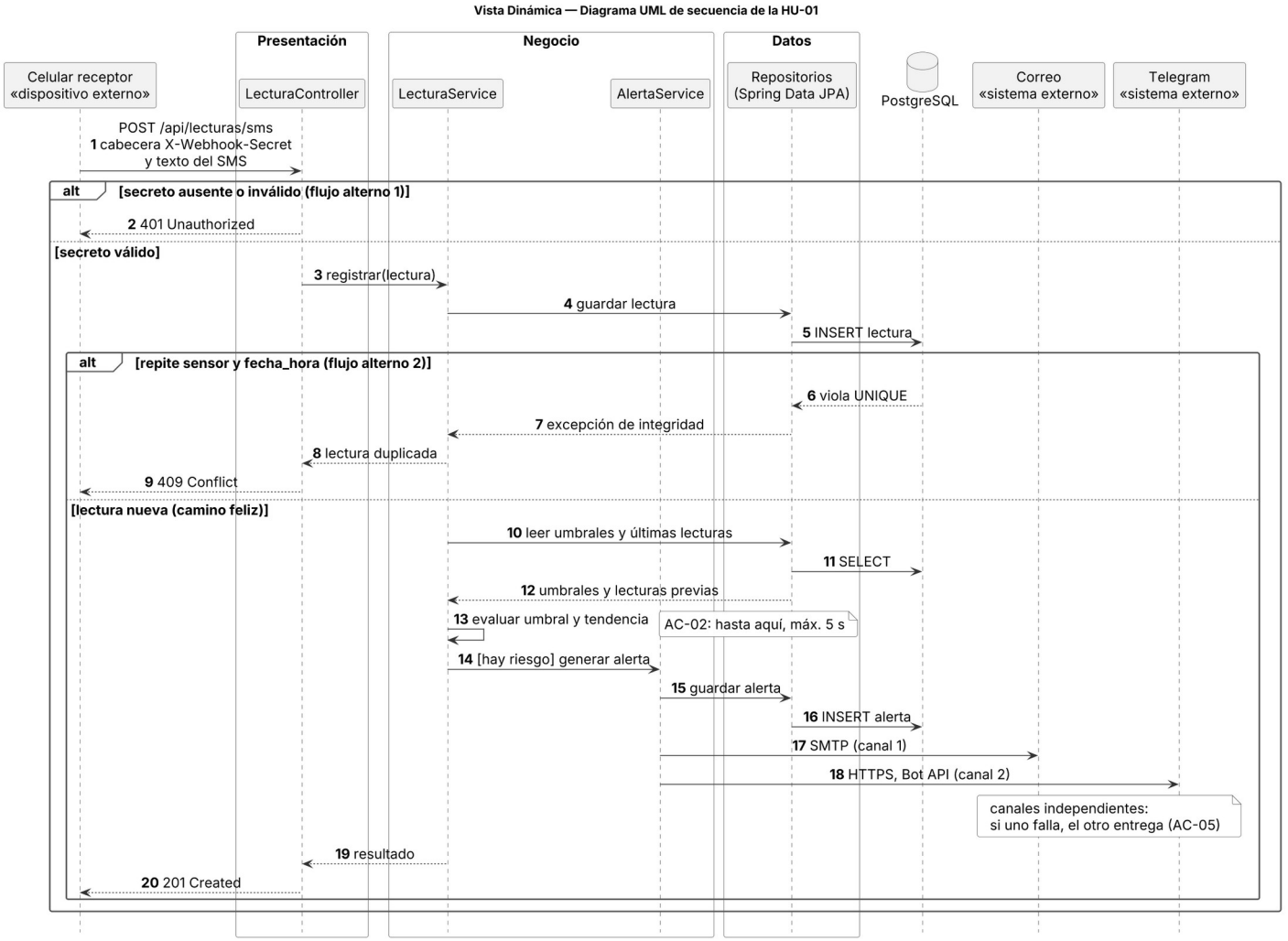


Diagrama UML de secuencia de la Vista Dinámica (HU-01)

Leyenda

Elemento del diagrama	Qué representa
Rectángulos arriba con una línea punteada hacia abajo	Participantes: quienes intervienen en el flujo. La línea punteada es su línea de vida y el tiempo corre de arriba hacia abajo. Los «dispositivo externo» y «sistema externo» son los mismos elementos externos de las vistas 3.1 y 3.2.
Recuadros «Presentación», «Negocio» y «Datos»	Las tres capas del patrón N-Capas (sección 2.1): el controlador, los servicios y los repositorios.
Cilindro PostgreSQL	La base de datos, el mismo contenedor de la Vista de Contenedores (3.2).
Flecha continua con número y flecha punteada	La continua es una llamada: el número indica el orden y la punta señala a quien la recibe. La punteada es la respuesta que vuelve a quien llamó.
Marco alt con condiciones entre corchetes	Alternativas excluyentes: ocurre solo una de ellas. Aquí son el flujo alterno 1 (401), el flujo alterno 2 (409) y el camino feliz.
Nota con un atributo de calidad	Indica la medida que debe cumplirse en ese punto del flujo (AC-02 y AC-05, sección 1.3).

Escenario (HU-01): la ejecuta el propio sistema cuando el celular receptor reenvía un SMS al webhook (sección 3.1). Como quien llama es un dispositivo y no una persona, el webhook se autentica con un secreto compartido en la cabecera X-Webhook-Secret, cuyo valor vive en una variable de entorno y nunca en el repositorio; si falta o no coincide, responde 401 (flujo alterno 1). Si la lectura ya existe para ese sensor y esa fecha y hora, la restricción UNIQUE de la base de datos la rechaza y responde 409 (flujo alterno 2). El CRUD de Alerta del corte vertical (sección 5) atraviesa las mismas capas: allí el 401 es por falta de token JWT y el 409 por la restricción UNIQUE (lectura_id, tipo).

Lectura en 30 segundos: un SMS llega como un POST al webhook con el secreto compartido. Si el secreto falla, responde 401 y termina (pasos 1 y 2). Si es válido, el controlador pasa la lectura al servicio, que la guarda en PostgreSQL; si ya existía, la base la rechaza por UNIQUE y el error sube por las capas hasta responder 409 (pasos 3 a 9). Si es nueva, el servicio lee umbrales y últimas lecturas y evalúa umbral y tendencia en máximo 5 s (AC-02, pasos 10 a 13). Si hay riesgo, se genera y guarda la alerta y se envía por correo y por Telegram, dos canales independientes (AC-05, pasos 14 a 18), y se responde 201 (pasos 19 y 20).

3.5 Vista de Despliegue — Diagrama UML de despliegue

Muestra dónde se ejecuta cada pieza del sistema: los equipos (nodos), el software que corre en cada uno, los puertos que escuchan y por dónde viaja la información entre ellos.

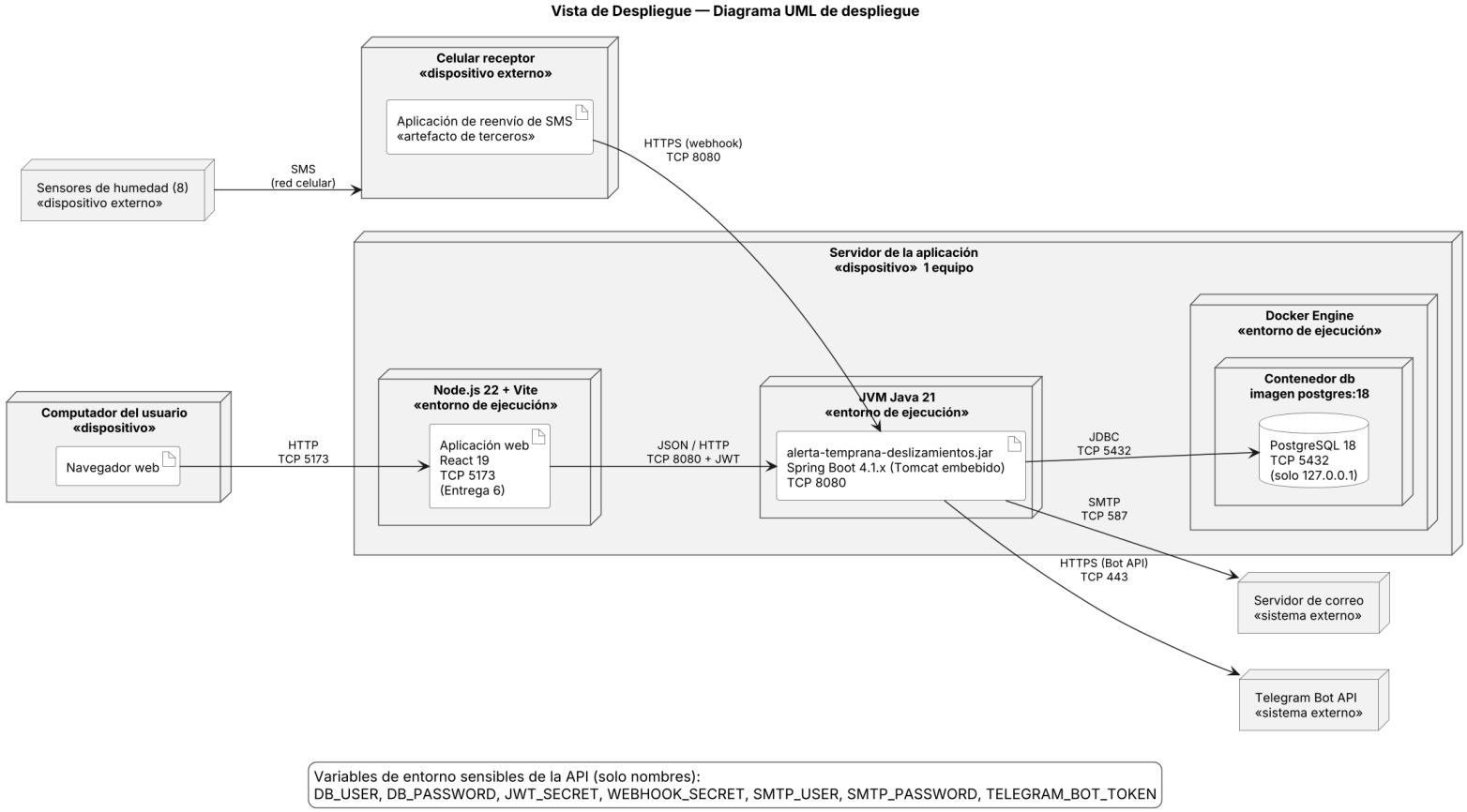


Diagrama UML de despliegue de la Vista de Despliegue

Leyenda

Elemento del diagrama	Qué representa
Cajas en 3D (nodos)	«dispositivo»: equipo físico. «entorno de ejecución»: software que ejecuta programas (JVM, Node.js, Docker).
Hoja con la esquina doblada y cilindro	Artefacto: archivo o programa que corre en un nodo (el .jar de la API, la aplicación web, la app de reenvío de SMS). El cilindro es la base de datos PostgreSQL 18.
«dispositivo externo» y «sistema externo»	Los mismos elementos externos de las vistas 3.1 y 3.2: no se controlan.
Flecha con etiqueta	Apunta hacia quien recibe la comunicación; la etiqueta indica el protocolo y el puerto TCP.
Recuadro inferior	Nombres de las variables de entorno sensibles; nunca sus valores.

Variables de entorno sensibles

Solo se escriben los nombres; los valores viven en el archivo .env, que Git ignora. El archivo .env.example trae los nombres con valores de ejemplo, incluidas las variables no secretas (host, puertos y nombre de la base, servidor SMTP y chat de Telegram).

Variable	Para qué sirve	Quién la lee	Estado
DB_USER y DB_PASSWORD	Usuario y clave de PostgreSQL.	La API y docker-compose.yml	En uso
JWT_SECRET	Clave con la que la API firma y verifica los tokens JWT (HS256) del administrador y del técnico; mínimo 32 caracteres.	La API (Spring Security)	En uso
WEBHOOK_SECRET	Secreto compartido que el celular envía en la cabecera X-Webhook-Secret (ADR-002).	La API y la app de reenvío	Pendiente
SMTP_USER y SMTP_PASSWORD	Credenciales del servidor de correo para enviar las alertas.	La API (canal de correo)	Pendiente
TELEGRAM_BOT_TOKEN	Token del bot de Telegram que envía las alertas.	La API (canal de Telegram)	Pendiente

Nodos, artefactos y puertos

Nodo	Artefacto	Puerto	Quién se comunica con él	Cómo se levanta (correspondencia)	Estado
Celular receptor (externo)	Aplicación de reenvío de SMS, de terceros y por elegir	—	Llama a la API por HTTPS (webhook)	Fuera de nuestro control (ADR-001); en desarrollo se simula con Postman	Externo
Servidor de la aplicación: JVM Java 21	alerta-temprana-deslizamientos.jar (Spring Boot 4.1.x con Tomcat embebido)	TCP 8080	Aplicación web, celular receptor (webhook) y Postman	Con el Maven Wrapper: mvnw spring-boot:run (README, paso 3), con application.yml y el archivo .env	En uso
Servidor de la aplicación: Docker Engine, contenedor db	PostgreSQL 18 (imagen postgres:18)	TCP 5432, solo en 127.0.0.1	Solo la API, por JDBC	Servicio db de docker-compose.yml: docker compose up -d db (README, paso 2); o PostgreSQL 18 instalado en el equipo	En uso
Servidor de la aplicación: Node.js 22 con Vite	Aplicación web React 19	TCP 5173	Navegador del usuario, por HTTP	npm run dev; se agrega con el corte vertical (aún no está en el repositorio)	Pendiente
Computador del usuario	Navegador web	—	Llama a la aplicación web	No se instala nada propio	Externo
Servidor de correo (externo)	Servicio SMTP	TCP 587	La API envía las alertas por SMTP	Se configura con SMTP_HOST, SMTP_PORT, SMTP_USER y SMTP_PASSWORD	Pendiente
Telegram Bot API (externo)	api.telegram.org	TCP 443	La API envía las alertas por HTTPS	Se configura con TELEGRAM_BOT_TOKEN y TELEGRAM_CHAT_ID	Pendiente

Correspondencia con el repositorio: el nodo de la base de datos es el servicio db de docker-compose.yml (imagen postgres:18, puerto 5432 publicado solo en 127.0.0.1 y volumen alerta_pgdata). La API se levanta con el Maven Wrapper y lee application.yml y el archivo .env; no exige Docker, porque basta con tener PostgreSQL 18 instalado y crear el rol y la base que indica el README (opción B). Los pasos completos de instalación y ejecución están en el README.md del repositorio. En la columna Estado, «En uso» es lo que corre hoy en el repositorio y «Pendiente», lo que está diseñado pero aún no se implementó: la aplicación React, el webhook de SMS y los canales de correo y Telegram. El puerto 8080 es el valor por defecto de SERVER_PORT; si está ocupado, esa variable lo cambia (por ejemplo, a 8081).

Lectura en 30 segundos: los 8 sensores envían sus SMS al celular receptor, cuya aplicación de reenvío los pasa por HTTPS al puerto 8080 de la API (ADR-001 y ADR-002). La API es un solo programa Java en el servidor: guarda en PostgreSQL 18 por el puerto 5432, que solo acepta conexiones del propio servidor, y envía las alertas por correo (puerto 587) y por Telegram (puerto 443). El técnico y el administrador usan la aplicación web (puerto 5173) desde su navegador, y esta habla con la API. Todo corre en un solo equipo, sin infraestructura adicional (RA-02).

Nota: en desarrollo el webhook se llama por HTTP en localhost, con Postman. En producción el SMS debe viajar por HTTPS, lo que exige un terminador TLS delante del puerto 8080; no está implementado y queda como deuda técnica (ADR-002). Además, la API y la base comparten equipo: un punto único de falla frente a la Confiabilidad (AC-05) que se evalúa en la Sección 6.

Sección 4: Registro de Decisiones de Arquitectura (ADRs)

Un ADR (Architecture Decision Record) registra una decisión de arquitectura: qué problema obligó a decidir, qué alternativas reales se consideraron, qué se decidió y qué se paga por ello. Los ADRs viven en docs/adr/ del repositorio y siguen la plantilla oficial: título, estatus, fecha, contexto, decisión y consecuencias. No se editan: si una decisión cambia, se escribe un ADR nuevo que reemplaza al anterior y el viejo queda como «Reemplazada por ADR-XXX».

El patrón N-Capas (2.1) y la pila tecnológica (2.2) ya están justificados en la Sección 2 con sus alternativas descartadas; los ADRs de esta sección registran las decisiones restantes del caso.

ID	Decisión	Estatus	Fecha	Atributos y restricciones
ADR-001	Recibir los SMS de los sensores mediante un webhook HTTP	Aceptada	24/10/2026	AC-02, AC-05, AC-07, RT-01, RA-02
ADR-002	Autenticar el webhook con un secreto compartido en la cabecera X-Webhook-Secret	Aceptada	24/10/2026	AC-06, AC-07, RA-02
ADR-003	Usar borrado lógico con un campo activo en todas las tablas	Aceptada	24/10/2026	AC-06, RF-03, RF-05, RF-09

4.1 ADR-001: Recibir los SMS de los sensores mediante un webhook HTTP

Campo	Contenido
Estatus	Aceptada
Fecha	24/10/2026
Contexto	Los 8 sensores de humedad envían sus lecturas por SMS, en lotes cada hora, a un celular que hoy un técnico revisa a mano (RT-02, RT-03). El hardware no se puede reemplazar ni modificar (RT-01) y no hay presupuesto para infraestructura adicional (RA-02). El caso no explica cómo pasa un SMS del celular a un sistema de software; es el supuesto que la sección 3.1 del DAS deja por confirmar con el cliente. **Fuerzas en juego:** ● AC-02 (Eficiencia de desempeño, prioridad Alta): el sistema debe procesar y evaluar cada lectura en 5 segundos o menos desde que la recibe. ● AC-05 (Confiabilidad, prioridad Alta): no hay turno de guardia nocturno (RA-03), así que la recepción no puede depender de que una persona revise el celular. ● AC-07 (Mantenibilidad, prioridad Alta): el mecanismo de entrada debe poder cambiar sin tocar las reglas de negocio ni el esquema de la base de datos. ● RA-02: no se puede gastar en infraestructura adicional. **Alternativas consideradas:**

Campo	Contenido
	1. **Webhook HTTP alimentado por el celular receptor actual.** Una aplicación de reenvío de SMS instalada en ese celular manda cada mensaje, como una petición HTTPS, a la API. 2. **Módem GSM USB conectado al servidor.** La SIM del celular pasaría al módem y la API leería los SMS con comandos AT. Obliga a comprar el módem y a mantener un puerto serial y su controlador en el servidor. 3. **Gateway de SMS de pago en la nube.** El proveedor entrega cada SMS por HTTP, pero los sensores tendrían que enviar al número del proveedor: habría que reconfigurar los 8 sensores, algo que el caso no permite asumir (RT-01), y se paga por mensaje (RA-02). 4. **Seguir con la revisión manual del técnico.** Se descarta: es el problema que el caso quiere resolver (RA-03).
Decisión	Decidimos recibir los SMS mediante un webhook HTTP (POST /api/lecturas/sms) al que el celular receptor existente reenvía cada mensaje, porque: ● mantiene el canal SMS y el número de los sensores sin tocar su hardware (RT-01, RT-02); ● no tiene costo, a diferencia del módem y del gateway (RA-02); ● hace que el SMS entre al sistema en el momento en que llega al celular, sin depender de una persona (RA-03), lo que permite cumplir los 5 segundos de AC-02 y la entrega sin guardia de AC-05; ● deja la recepción aislada en un solo controlador: si después se pasa a módem o gateway, solo cambia quién llama a ese endpoint y no las reglas de negocio (AC-07).
Consecuencias positivas	● Costo cero: no se compra ni se instala hardware. ● En desarrollo el SMS se simula con Postman usando el mismo contrato HTTP que usará el celular. ● Los reintentos son seguros: si la aplicación de reenvío manda dos veces el mismo mensaje, la restricción UNIQUE (sensor_id, fecha_hora) rechaza el segundo con 409 y no se duplica la lectura. ● Cambiar de mecanismo de entrada no afecta la capa de negocio.
Consecuencias negativas y deuda técnica asumida	● El celular receptor sigue siendo un punto único de falla (batería, cobertura, internet, aplicación cerrada por Android). Esta decisión no lo elimina. Mitigación parcial: la consulta de estado de sensores (HU-03) muestra qué sensor dejó de reportar. ● Depende de una aplicación de terceros instalada en el celular, cuyos reintentos y formato de envío no controlamos. ● Si la API está caída cuando llega el SMS y la aplicación no reintenta, la lectura se pierde: no hay cola de mensajes ni broker (descartados en la sección 2.1 por costo y por puntos de falla adicionales). ● Supuestos sin confirmar con el cliente: que el celular admita una aplicación de reenvío y tenga conexión a internet, y cuál es el formato exacto del SMS (sensor, fecha y hora, humedad). Si el cliente no los confirma, este ADR se reemplaza por uno nuevo (módem o gateway). ● La API queda expuesta a internet, por lo que el webhook necesita autenticación (ADR-002) y HTTPS en producción.

4.2 ADR-002: Autenticar el webhook con un secreto compartido en la cabecera X-Webhook-Secret

Campo	Contenido
Estatus	Aceptada
Fecha	24/10/2026
Contexto	El webhook del ADR-001 no lo llama una persona sino un dispositivo (el celular receptor con su aplicación de reenvío). El resto de la API se protege con JWT, que se obtiene iniciando sesión con usuario y contraseña y vence; un dispositivo desatendido no puede hacer ese inicio de sesión. Si el webhook no se protegiera, cualquiera que conozca la URL podría enviar lecturas falsas: provocar una alerta falsa que desgaste la confianza en el sistema o esconder una lectura real detrás de datos inventados. La Seguridad (AC-06) tiene prioridad Baja en la Ficha de Drivers, así que la protección debe ser de bajo costo, pero la consecuencia de un dato falso sobre riesgo de vida es grave. **Fuerzas en juego:** ● AC-06 (Seguridad): solo el celular autorizado puede inyectar lecturas. ● AC-07 (Mantenibilidad): cambiar o rotar la credencial no debe exigir cambios de código. ● RA-02 y la restricción de tiempo (sección 1.4): sin software ni servicios adicionales. ● Supuesto: la aplicación de reenvío permite configurar la URL y cabeceras HTTP fijas, pero no calcular firmas. **Alternativas consideradas:** 1. **Secreto compartido estático en una cabecera HTTP.** El celular lo envía en cada petición y la API lo compara con el valor guardado en una variable de entorno. 2. **JWT como el de las personas.** El token vence y para renovarlo hay que iniciar sesión; la aplicación de reenvío no puede. Un JWT de larga duración sería, en la práctica, otro secreto estático pero más complejo. 3. **Firma HMAC del cuerpo con marca de tiempo.** Es más segura porque una petición capturada no se puede repetir, pero la aplicación de reenvío tendría que calcular la firma: habría que programar una aplicación propia o un intermediario. 4. **Lista blanca de direcciones IP.** El celular sale por datos móviles con una IP que cambia; no sirve. 5. **Sin autenticación.** Se descarta por el riesgo descrito arriba.
Decisión	Decidimos autenticar el webhook con un secreto compartido largo y aleatorio (al menos 32 caracteres) enviado en la cabecera X-Webhook-Secret, porque es lo único que una aplicación de reenvío común puede mandar sin código propio (restricción de tiempo y RA-02) y protege la entrada contra lecturas falsas (AC-06). El valor vive en la variable de entorno WEBHOOK_SECRET y nunca en el repositorio; la API lo compara en tiempo constante y responde 401 si la cabecera falta o no coincide (flujo alterno 1 de la sección 3.4). Rotarlo es cambiar la variable y reconfigurar la aplicación del celular, sin tocar el código (AC-07).

Campo	Contenido
Consecuencias positivas	• Implementación mínima: una comparación antes de procesar el mensaje. • Fácil de verificar con Postman: sin cabecera o con cabecera errónea responde 401; con la cabecera correcta responde 201. • Rotación sin cambios de código.
Consecuencias negativas y deuda técnica asumida	• Es una credencial estática: si se filtra, sirve hasta que se rote. • Solo es segura sobre HTTPS; en HTTP viaja en claro. En desarrollo (localhost) se usa HTTP y en producción se debe poner un terminador TLS (proxy inverso o certificado). Eso aún no está implementado en el esqueleto. • No impide que alguien con una petición capturada la repita. Lo mitiga la restricción UNIQUE (sensor_id, fecha_hora): repetir la misma lectura devuelve 409 y no la duplica, pero una lectura nueva falsificada necesita el secreto. • Todos los dispositivos comparten el mismo secreto: si algún día hubiera más de un celular receptor, no se distinguiría cuál envió el mensaje. • Si la aplicación de reenvío que se elija no permite cabeceras personalizadas, este ADR se reemplaza (por ejemplo, el secreto en la URL o en el cuerpo).

4.3 ADR-003: Usar borrado lógico con un campo activo en todas las tablas

Campo	Contenido
Estatus	Aceptada
Fecha	24/10/2026
Contexto	Cuando el sistema "elimina" un registro (un sensor retirado, un umbral que se deja de usar, un usuario que sale) hay que decidir si la fila desaparece o queda marcada. La plantilla del curso exige borrado lógico en toda entidad de negocio (sección 5.2), pero además hay razones propias del caso: • La detección de tendencia (RF-05) necesita el histórico de lecturas de cada sensor (RF-03); borrarlas debilita el análisis. • La trazabilidad de quién y cuándo cambió un umbral (HU-02, AC-06) y el historial de alertas emitidas (RF-09) son evidencia útil ante un deslizamiento. • Los errores humanos de borrado deben poder deshacerse. **Alternativas consideradas:** 1. **Borrado físico (DELETE).** Es lo más simple y no deja filas inactivas, pero se pierde el historial y no se puede deshacer. 2. **Borrado lógico con una columna activo BOOLEAN NOT NULL DEFAULT TRUE.** La fila se conserva y queda marcada como inactiva. 3. **Borrado físico más una tabla de archivo o auditoría.** Conserva el historial en otra tabla, pero exige más tablas y disparadores (triggers) para copiar cada fila antes de borrarla.
Decisión	Decidimos usar borrado lógico con la columna activo en las cinco tablas (usuario, sensor, lectura, umbral y alerta), porque conserva el histórico que alimenta la tendencia (RF-03, RF-05) y la trazabilidad (RF-09, AC-06),

Campo	Contenido
	permite reactivar un registro borrado por error y cumple la sección 5.2 de la plantilla. Las llaves foráneas usan ON DELETE RESTRICT, de modo que un borrado físico accidental de una fila referenciada también falla. En la API, la operación DELETE hace un UPDATE ... SET activo = FALSE y responde 204.
Consecuencias positivas	• El historial de lecturas y alertas se conserva completo. • Un sensor o umbral desactivado se puede reactivar sin perder su historia. • La prueba del corte vertical es directa: DELETE devuelve 204 y la fila sigue en la base con activo = FALSE.
Consecuencias negativas y deuda técnica asumida	• Toda consulta debe filtrar activo = TRUE; olvidarlo muestra datos "borrados". Se centraliza en los repositorios para que no dependa de cada consulta. El índice parcial idx_alerta_activas_recientes (WHERE activo) ya atiende la consulta del técnico (HU-03). • Las restricciones UNIQUE siguen contando las filas inactivas: un umbral desactivado no se puede volver a crear con el mismo sensor y nivel, se reactiva o se edita. Alternativa si estorbara: un índice único parcial (WHERE activo), con la migración correspondiente. • La base crece sin límite. Con una lectura por sensor por hora serían unas 70 000 filas al año en lectura (8 sensores por 24 horas por 365 días), un volumen pequeño; si creciera, habría que definir una política de archivo. • Los datos de los usuarios no se eliminan físicamente. El caso no identifica normativa de protección de datos (sección 1.4); si apareciera una, habría que agregar anonimización o borrado físico para esa tabla.

Sección 5: Entregable de Programación Ejecutable (Corte Vertical)

Esta sección demuestra que la arquitectura funciona con un CRUD completo de la entidad Alerta, protegido con JWT. Reúne el repositorio y el esqueleto de la API (5.1), el script de base de datos (5.2), el CRUD con su seguridad (5.3) y las pruebas (5.4).

5.1 Repositorio y esqueleto de la API

El repositorio alerta-temprana-deslizamientos en GitHub reúne el DAS, los ADRs, las migraciones y el esqueleto de la API. Como la pila es Java con Spring Boot (2.2), la estructura que propone la plantilla se adapta a las convenciones de Maven: lo evaluable es la separación de capas, no el lenguaje.

Ruta	Contenido
docs/	DAS en Word y PDF; adr/ con los registros de decisiones (Sección 4); diagramas/ con las fuentes .puml y los .png.
db/migrations/ y db/pruebas/	V1__esquema_inicial.sql, la migración que Flyway aplica al arrancar (5.2). En db/pruebas/: las pruebas de integridad del esquema, los datos de demostración (datos_demo.sql y usuarios_demo.sql) y reiniciar_alertas.sql, que vacía las alertas entre una ejecución de pruebas y otra (5.4).
src/main/java/com/ deslizamientos/alerta/	Código de la API con una carpeta por capa del patrón N-Capas: controller y dto (presentación), service y mapper (negocio), repository y model (datos). Aparte, exception (manejo centralizado de errores) y security (token y roles), que atraviesan las capas.
src/main/resources/ application.yml	Configuración de la API; los valores sensibles se leen de variables de entorno.
src/test/java/	Pruebas unitarias con JUnit y Mockito: 10 pruebas (5.4).
docker-compose.yml y .env.example	Base PostgreSQL 18 de desarrollo y nombres de las variables de entorno (3.5).
postman/	Colección Postman del corte vertical: 16 peticiones con 32 pruebas automáticas (5.4).

Esqueleto corriendo y conexión a la base de datos verificada: al arrancar, la API se conecta a PostgreSQL y Flyway aplica la migración V1. La petición GET /api/estado responde 200 con el estado UP, el motor y la versión de PostgreSQL, el nombre de la base y tablasDelEsquema igual a 5. Si la base está apagada, responde 503 y /actuator/health responde DOWN. EstadoController solo llama a EstadoService y este solo a BaseDatosRepository, el mismo recorrido por capas de la Vista Dinámica (3.4); los errores se traducen en un solo lugar, ManejadorGlobalDeErrores. Con la seguridad de la sección 5.3, /api/estado y /actuator/health siguen siendo públicos. Los pasos para reproducirlo están en el README.md.

5.2 Script SQL DDL y migraciones

El script crea las cinco tablas del diagrama de clases (3.3) en PostgreSQL 18: cada clase es una tabla y cada atributo, una columna con el mismo nombre. Está en db/migrations/V1__esquema_inicial.sql y Flyway lo aplica al arrancar la API (sección 2.2). El nombre sigue el formato que exige Flyway: la letra V, el número de versión, dos guiones bajos y la descripción.

Del tipo de la clase al tipo de la columna

Tipo en el diagrama (Java)	Tipo en el script (PostgreSQL)	Dónde se usa
Long	BIGINT GENERATED ALWAYS AS IDENTITY (id) o BIGINT (llaves foráneas)	PostgreSQL genera el id; la llave foránea guarda el id de la otra tabla.
String	VARCHAR(n)	n es el largo máximo de la columna, por ejemplo 20 en codigo.
BigDecimal	NUMERIC(5,2)	Número decimal exacto con 2 decimales, hasta 100,00: humedad y valor.
OffsetDateTime	TIMESTAMPTZ	Fecha y hora con zona horaria: fecha_hora, recibida_en, generada_en.
Boolean	BOOLEAN	activo: TRUE es vigente y FALSE es borrado lógico.

Restricciones de integridad por tabla

Tabla	PK	FK (REFERENCES)	UNIQUE	CHECK
usuario	id	—	username	rol en {ADMINISTRADOR, TECNICO}
sensor	id	—	codigo	—
lectura	id	sensor_id → sensor(id)	(sensor_id, fecha_hora)	humedad entre 0 y 100
umbral	id	sensor_id → sensor(id); modificado_por → usuario(id)	(sensor_id, nivel)	nivel en {ADVERTENCIA, CRITICO}; valor entre 0 y 100
alerta	id	lectura_id → lectura(id)	(lectura_id, tipo)	tipo en {UMBRAL, TENDENCIA}; nivel en {ADVERTENCIA, CRITICO}

Borrado lógico: las cinco tablas tienen la columna activo (BOOLEAN, por defecto TRUE) y ninguna fila se borra físicamente. Por eso las llaves foráneas usan ON DELETE RESTRICT: un borrado físico de una fila que tiene hijos falla en la base de datos. Como las filas desactivadas siguen existiendo, los UNIQUE las siguen contando: un umbral desactivado se reactiva o se edita, no se crea otro igual.

Índices

Los tres UNIQUE compuestos ya crean un índice cuya primera columna es la llave foránea, así que sensor_id y lectura_id quedan indexadas sin índices adicionales. Solo hace falta uno más.

Índice	Consulta que atiende	Justificación
uq_lectura_sensor_fecha (sensor_id, fecha_hora)	Las últimas lecturas de un sensor, para detectar tendencia.	RF-05 y AC-02: la evaluación debe terminar en máximo 5 s.
uq_umbral_sensor_nivel (sensor_id, nivel)	Los umbrales de un sensor al evaluar una lectura.	HU-01: se consulta con cada lectura que llega.
uq_alerta_lectura_tipo (lectura_id, tipo)	Las alertas que salieron de una lectura.	Evita duplicados y acelera la consulta por lectura.
idx_alerta_activas_recientes (generada_en DESC) WHERE activo	La lista de alertas activas, de la más reciente a la más antigua.	HU-03: es la consulta del técnico. Es parcial (solo activas) para que sea más pequeño.

No se crea índice sobre umbral.modificado_por: solo sirve para saber quién cambió un límite y la tabla tiene pocas filas (8 sensores por 2 niveles).

Nota: el rango de 0 a 100 en humedad y valor supone que el sensor reporta la humedad como porcentaje. El caso no indica la unidad ni el formato del SMS (supuestos pendientes de la sección 1); si el cliente confirma otra unidad, el CHECK se ajusta en una nueva migración.

Pruebas de integridad

Con las tablas creadas y datos válidos cargados (un usuario administrador, un sensor, un umbral, una lectura y una alerta), estas pruebas confirman que la base de datos hace cumplir las reglas. Están en db/pruebas/pruebas_integridad.sql y se pueden ejecutar en vivo.

Prueba	Resultado en la base de datos
Insertar un usuario con rol INVITADO	Rechazado: viola el CHECK ck_usuario_rol.
Insertar una lectura con humedad 150	Rechazado: viola el CHECK ck_lectura_humedad.
Insertar dos veces la lectura del mismo sensor y la misma hora	Rechazado: viola uq_lectura_sensor_fecha (en la API se convierte en 409).
Insertar una lectura de un sensor que no existe	Rechazado: viola la llave foránea fk_lectura_sensor.
Insertar una alerta con tipo OTRO	Rechazado: viola el CHECK ck_alerta_tipo.
Insertar dos veces el mismo tipo de alerta para una lectura	Rechazado: viola uq_alerta_lectura_tipo.
Insertar un sensor con un código repetido	Rechazado: viola uq_sensor_codigo.
Borrar físicamente un sensor que tiene lecturas	Rechazado: la llave foránea con ON DELETE RESTRICT lo impide.
Desactivar un sensor (activo = FALSE)	Aceptado: la fila sigue en la tabla, marcada como inactiva.

Script completo (db/migrations/V1__esquema_inicial.sql)

```
-- Migración inicial del esquema de base de datos
-- Sistema de Alerta Temprana de Deslizamientos (Caso 7)
-- Motor: PostgreSQL 18. Migraciones versionadas con Flyway.
--
-- Este script materializa el diagrama de clases de la sección 3.3 del DAS:
-- cada clase es una tabla y cada atributo es una columna con el mismo nombre.
-- Integridad: PK en toda tabla, FK con REFERENCES, CHECK y UNIQUE.
-- Borrado lógico: toda tabla tiene la columna "activo" (nunca se borran filas).
-- Las FK usan ON DELETE RESTRICT: un borrado físico de una fila con hijos falla.


-- Usuario: quien entra al sistema con JWT. El rol define sus permisos (HU-02, HU-03).
CREATE TABLE usuario (
    id             BIGINT GENERATED ALWAYS AS IDENTITY,
    username       VARCHAR(50)  NOT NULL,
    password_hash  VARCHAR(100) NOT NULL,
    rol            VARCHAR(15)  NOT NULL,
    activo         BOOLEAN      NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_usuario          PRIMARY KEY (id),
    CONSTRAINT uq_usuario_username UNIQUE (username),
    CONSTRAINT ck_usuario_rol      CHECK (rol IN ('ADMINISTRADOR', 'TECNICO'))
);

-- Sensor: cada uno de los 8 sensores de humedad (HU-03, RF-10).
CREATE TABLE sensor (
    id         BIGINT GENERATED ALWAYS AS IDENTITY,
    codigo     VARCHAR(20)  NOT NULL,
    ubicacion  VARCHAR(100) NOT NULL,
    activo     BOOLEAN      NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_sensor        PRIMARY KEY (id),
    CONSTRAINT uq_sensor_codigo UNIQUE (codigo)
);

-- Lectura: cada medición de humedad recibida por SMS; es el histórico (HU-01, RF-03).
-- El UNIQUE sobre (sensor_id, fecha_hora) impide registrar dos veces la misma
-- medición: es el origen del 409 cuando el SMS llega repetido.
CREATE TABLE lectura (
    id           BIGINT GENERATED ALWAYS AS IDENTITY,
    sensor_id    BIGINT       NOT NULL,
    humedad      NUMERIC(5,2) NOT NULL,
    fecha_hora   TIMESTAMPTZ  NOT NULL,
    recibida_en  TIMESTAMPTZ  NOT NULL DEFAULT now(),
    activo       BOOLEAN      NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_lectura        PRIMARY KEY (id),
    CONSTRAINT fk_lectura_sensor FOREIGN KEY (sensor_id)
        REFERENCES sensor (id) ON DELETE RESTRICT,
    CONSTRAINT uq_lectura_sensor_fecha UNIQUE (sensor_id, fecha_hora),
    CONSTRAINT ck_lectura_humedad CHECK (humedad BETWEEN 0 AND 100)
);

-- Umbral: límite de riesgo de un sensor, por nivel (HU-02, RF-08).
-- Un sensor tiene como máximo un umbral por nivel. Para cambiar un límite se
-- edita el valor (o se reactiva el umbral); no se crea uno repetido.
CREATE TABLE umbral (
    id              BIGINT GENERATED ALWAYS AS IDENTITY,
    sensor_id       BIGINT       NOT NULL,
    nivel           VARCHAR(12)  NOT NULL,
    valor           NUMERIC(5,2) NOT NULL,
    modificado_por  BIGINT       NOT NULL,
    modificado_en   TIMESTAMPTZ  NOT NULL DEFAULT now(),
    activo          BOOLEAN      NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_umbral        PRIMARY KEY (id),
    CONSTRAINT fk_umbral_sensor FOREIGN KEY (sensor_id)
        REFERENCES sensor (id) ON DELETE RESTRICT,
    CONSTRAINT fk_umbral_usuario FOREIGN KEY (modificado_por)
        REFERENCES usuario (id) ON DELETE RESTRICT,
    CONSTRAINT uq_umbral_sensor_nivel UNIQUE (sensor_id, nivel),
    CONSTRAINT ck_umbral_nivel CHECK (nivel IN ('ADVERTENCIA', 'CRITICO')),
    CONSTRAINT ck_umbral_valor CHECK (valor BETWEEN 0 AND 100)
);
```

```
-- Alerta: lo que se genera cuando una lectura supera un umbral o muestra
-- tendencia de riesgo (HU-01, RF-06, RF-09). Es la entidad del CRUD del corte vertical.
-- El UNIQUE sobre (lectura_id, tipo) evita generar dos veces el mismo tipo de
-- alerta para la misma lectura: es el origen del 409 en un POST repetido.
CREATE TABLE alerta (
    id           BIGINT GENERATED ALWAYS AS IDENTITY,
    lectura_id   BIGINT       NOT NULL,
    tipo         VARCHAR(10)  NOT NULL,
    nivel        VARCHAR(12)  NOT NULL,
    mensaje      VARCHAR(255) NOT NULL,
    generada_en  TIMESTAMPTZ  NOT NULL DEFAULT now(),
    activo       BOOLEAN      NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_alerta         PRIMARY KEY (id),
    CONSTRAINT fk_alerta_lectura FOREIGN KEY (lectura_id)
        REFERENCES lectura (id) ON DELETE RESTRICT,
    CONSTRAINT uq_alerta_lectura_tipo UNIQUE (lectura_id, tipo),
    CONSTRAINT ck_alerta_tipo  CHECK (tipo IN ('UMBRAL', 'TENDENCIA')),
    CONSTRAINT ck_alerta_nivel CHECK (nivel IN ('ADVERTENCIA', 'CRITICO'))
);

-- Índices justificados.
-- Los UNIQUE de lectura (sensor_id, fecha_hora), umbral (sensor_id, nivel) y
-- alerta (lectura_id, tipo) ya crean un índice cuya primera columna es la FK, así
-- que las FK sensor_id y lectura_id quedan indexadas sin índices adicionales.
-- El de lectura también atiende la consulta de las últimas lecturas de un sensor
-- (tendencia, RF-05), que debe cumplir el máximo de 5 s (AC-02).
-- Solo falta uno: la lista de alertas activas, de la más reciente a la más antigua,
-- que consulta el técnico (HU-03). Es parcial (solo activas) para que sea más pequeño.
CREATE INDEX idx_alerta_activas_recientes
    ON alerta (generada_en DESC)
    WHERE activo;
```

5.3 CRUD de Alerta con autenticación JWT

El corte vertical atraviesa las tres capas del patrón N-Capas (2.1) con una sola entidad, Alerta (HU-01). La petición entra por el controlador, el servicio aplica las reglas, el repositorio guarda en PostgreSQL y la respuesta vuelve por el mismo camino de la Vista Dinámica (3.4). La seguridad actúa antes: es un filtro que revisa el token y el rol sin que el controlador intervenga.

Qué hace cada capa

Capa	Paquetes y clases	Responsabilidad
Presentación	controller: AlertaController, AuthController. dto: AlertaCrearDto, AlertaActualizarDto, AlertaDto, LoginDto, TokenDto.	Recibe el JSON, lo valida con @Valid y responde el código HTTP. No tiene reglas de negocio ni SQL.
Negocio	service: AlertaService, AuthService. mapper: AlertaMapper.	Aplica las reglas: la lectura debe existir (400), no se repite un tipo de alerta para la misma lectura (409) y el borrado es lógico. No conoce HTTP: lanza excepciones. Cada operación es una transacción.
Datos	model: Alerta, Lectura, Usuario, TipoAlerta, NivelAlerta, Rol. repository: AlertaRepository, LecturaRepository, UsuarioRepository.	Las entidades JPA se mapean a las tablas de la migración V1 (5.2): Flyway crea las tablas e Hibernate no las modifica (ddl-auto: none).
Transversal	exception: ManejadorGlobalDeErrores. security: SeguridadConfig, RespuestasDeSeguridad.	Traduce las excepciones a respuestas JSON con una sola forma y valida el token y el rol antes del controlador.

Endpoints, permisos y respuestas

Método y ruta	Quién puede	Éxito	Errores
POST /api/auth/login	Cualquiera	200 con el token	400 si falta un dato; 401 si el usuario o la contraseña son incorrectos
POST /api/alertas	ADMINISTRADOR	201 con la alerta y la cabecera Location	400 si los datos son inválidos o la lectura no existe; 401; 403; 409 si ya existe una alerta de ese tipo para la lectura
GET /api/alertas	ADMINISTRADOR y TECNICO	200 con las alertas activas, de la más reciente a la más antigua	401
GET /api/alertas/{id}	ADMINISTRADOR y TECNICO	200 con la alerta	401; 404 si no existe o fue borrada
PUT /api/alertas/{id}	ADMINISTRADOR	200 con la alerta; solo cambian el nivel y el mensaje	400; 401; 403; 404
DELETE /api/alertas/{id}	ADMINISTRADOR	204 sin cuerpo (borrado lógico)	401; 403; 404
GET /api/estado y /actuator/health	Cualquiera	200 con el estado UP	503 si la base de datos no responde

Todos los errores tienen la misma forma, un código y un mensaje; por ejemplo, un POST repetido responde:

```
{"estado":"CONFLICT","mensaje":"Ya existe una alerta UMBRAL para la lectura 1"}
```

Los códigos son BAD_REQUEST, UNAUTHORIZED, FORBIDDEN, NOT_FOUND, CONFLICT y DOWN. ManejadorGlobalDeErrores traduce en un solo lugar las excepciones del servicio. El 401 y el 403 los escribe RespuestasDeSeguridad, porque ocurren en un filtro, antes de que la petición llegue a un controlador. En las rutas con {id}, un id que no es un número responde 400.

Borrado lógico (ADR-003)

DELETE no borra la fila: ejecuta UPDATE alerta SET activo = FALSE y responde 204. Las consultas de lectura (listar y obtener) solo ven filas con activo = TRUE, así que una alerta borrada desaparece de la lista y su GET responde 404, mientras la fila sigue en la base de datos. Como el UNIQUE (lectura_id, tipo) también cuenta las filas inactivas, volver a crear una alerta del mismo tipo para la misma lectura responde 409 aunque la anterior esté borrada; esa consecuencia se evalúa en la Sección 6 (R-05).

Autenticación y roles con JWT (HU-02, HU-03)

El usuario inicia sesión en POST /api/auth/login con su nombre y su contraseña. AuthService compara la contraseña con el hash BCrypt guardado en la tabla usuario y, si coincide, devuelve un token JWT firmado con HS256 que vence a los 60 minutos (JWT_EXPIRACION_MINUTOS) y lleva el rol en el claim «rol». En cada petición siguiente el cliente envía el token en la cabecera Authorization: Bearer; la API lo valida sin guardar sesión y aplica el permiso del rol. Un usuario inexistente y una contraseña errónea reciben el mismo mensaje, para no revelar qué usuarios existen.

Rol	Permisos
ADMINISTRADOR	Consultar, crear, modificar y borrar alertas.
TECNICO	Solo consultar alertas (GET). Un POST, PUT o DELETE responde 403.
Sin token válido	Cualquier ruta protegida responde 401: falta el token, está vencido o no es válido.

El secreto de firma se lee de la variable JWT_SECRET (3.5) y debe tener al menos 32 caracteres: si falta o es más corto, la API no arranca. El script db/pruebas/usuarios_demo.sql crea un usuario por rol, solo para la demostración y las pruebas; sus contraseñas se guardan como hash BCrypt, nunca en texto.

Recorrido de la demostración

Es el recorrido que se muestra en vivo. Parte de dos scripts de la carpeta db/pruebas/: datos_demo.sql, con un sensor y dos lecturas, y usuarios_demo.sql, con un usuario por rol. Los pasos están en el README.md.

Paso	Petición	Resultado
1	POST /api/auth/login con el usuario administrador.	200 con el token.
2	POST /api/alertas con la cabecera Authorization: Bearer y el cuerpo {"lecturaId": 1, "tipo": "UMBRAL", "nivel": "CRITICO", "mensaje": "Humedad sobre el umbral crítico"}.	201 con la alerta y su cabecera Location.
3	GET /api/alertas/{id} con el id recibido.	200 con la misma alerta.
4	En la base de datos: SELECT id, lectura_id, tipo, nivel, activo FROM alerta;	La fila nueva, con activo en verdadero.
5	Repetir el POST del paso 2.	409 CONFLICT.

5.4 Pruebas del corte vertical

Las pruebas comprueban lo que afirma la sección 5.3 en dos niveles: pruebas unitarias de la capa de negocio, sin base de datos ni HTTP, y una colección Postman con pruebas automáticas contra la API en marcha.

Pruebas unitarias (src/test/java)

Clase	Pruebas	Qué comprueban
AlertaServiceTest	5	Crear con datos válidos guarda y devuelve la alerta; una lectura que no existe es 400; una alerta repetida para la misma lectura y tipo es 409; obtener una alerta inexistente es 404; eliminar hace borrado lógico y no borra la fila.
AuthServiceTest	3	Con credenciales correctas devuelve un token con el rol; con contraseña incorrecta o con un usuario inexistente lanza el error que se convierte en 401 y no genera token.
EstadoServiceTest	2	El estado es UP con las cinco tablas del esquema y DEGRADADO si faltan tablas (esqueleto, 5.1).

Los repositorios y el codificador de tokens son dobles de Mockito; el BCrypt es el real, para comprobar que la contraseña se compara contra el hash. Resultado: ./mvnw test ejecuta las 10 pruebas sin fallos (BUILD SUCCESS).

Colección Postman (postman/alerta-temprana.postman_collection.json)

La colección tiene 16 peticiones y 32 pruebas automáticas que verifican el código HTTP y el contenido de la respuesta. Se ejecutan en orden: las primeras guardan el token y el id de la alerta en variables de la colección y las siguientes los usan.

#	Petición	Esperado	Qué demuestra
01	Login administrador	200	Devuelve el token y el rol ADMINISTRADOR.
02	Login con clave incorrecta	401	No se emite token (UNAUTHORIZED).
03	Listar alertas sin token	401	La ruta está protegida.
04	Crear alerta como administrador	201	Crea la alerta y devuelve su Location.
05	Crear la misma alerta otra vez	409	UNIQUE (lectura_id, tipo): CONFLICT.
06	Crear alerta con datos inválidos	400	Validación del cuerpo: BAD_REQUEST.
07	Crear alerta de una lectura que no existe	400	Regla de negocio del servicio.
08	Obtener la alerta por id	200	Recupera la alerta creada.
09	Listar alertas	200	La lista incluye la alerta creada.
10	Actualizar nivel y mensaje	200	El PUT cambia solo el nivel y el mensaje.
11	Login técnico	200	Token con el rol TECNICO.
12	Crear alerta como técnico	403	El rol TECNICO no puede escribir: FORBIDDEN.

#	Petición	Esperado	Qué demuestra
13	Consultar alerta como técnico	200	El rol TECNICO sí puede leer.
14	Eliminar la alerta	204	Borrado lógico, sin cuerpo.
15	Obtener la alerta eliminada	404	La alerta borrada ya no se ve: NOT_FOUND.
16	Estado de la API sin token	200	La ruta pública responde sin token.

Cómo ejecutarla

Con la base y la API en marcha y los datos de demostración cargados, se importa el archivo en Postman, se ajusta la variable baseUrl si la API no usa el puerto de la colección (http://localhost:8081) y se ejecuta con Collection Runner. Antes de repetir la ejecución se vacían las alertas con db/pruebas/reiniciar_alertas.sql: el UNIQUE cuenta las alertas borradas y, sin ese paso, la petición 04 respondería 409 (R-05 en la Sección 6).

Estado de la evidencia: las 10 pruebas unitarias pasan, y el CRUD y los roles se probaron por consola contra la API real. El resultado de la colección ejecutada en Postman contra la API real queda por registrar (captura del Collection Runner con 16 de 16 peticiones y 32 de 32 pruebas en verde).

Sección 6: Evaluación de Arquitectura y Trade-offs

Toda arquitectura sacrifica algo. Esta sección declara lo que se priorizó y lo que se pagó por ello (6.1), los puntos donde un cambio pequeño afecta un atributo de calidad (6.2) y si la arquitectura entregada satisface los escenarios de la sección 1.3 (6.3).

6.1 Análisis de compromisos

Atributo priorizado	Atributo sacrificado	Evidencia del compromiso	¿Por qué es aceptable en este proyecto?
Mantenibilidad (AC-07) y Eficiencia de desempeño (AC-02): N-Capas en una sola aplicación (2.1).	Independencia del núcleo respecto del framework (lo que daría una arquitectura Hexagonal).	Las entidades llevan anotaciones de JPA y los servicios usan Spring (@Service, @Transactional); la entrada de SMS no está detrás de un puerto intercambiable.	Un solo desarrollador y plazo corto (1.4). El cambio de entrada previsto, otro canal, entra por HTTP al mismo servicio (ADR-001) sin tocar el núcleo.
Costo (RA-02) y hardware intacto (RT-01): webhook HTTP con un celular que reenvía los SMS (ADR-001).	Confiabilidad de la entrada (AC-05).	El celular receptor es el único punto de entrada (RT-03), la aplicación de reenvío es de terceros y sus supuestos no están confirmados con el cliente.	No hay presupuesto para un gateway de pago ni se pueden tocar los sensores. El riesgo se reduce detectando sensores en silencio (R-02).
Simplicidad de seguridad y despliegue: JWT sin sesiones, un solo secreto HS256 y secreto compartido para el webhook (ADR-002).	Revocación inmediata de un token y aislamiento de credenciales.	Un token no se puede anular antes de vencer (60 minutos) y el mismo JWT_SECRET firma y valida todos los tokens (SeguridadConfig).	Hay dos roles y pocos usuarios en un solo servidor sin infraestructura adicional (RA-02); el vencimiento corto limita el daño de un token filtrado (R-04).
Trazabilidad e integridad del historial: borrado lógico en todas las tablas (ADR-003).	Simplicidad de las consultas y del UNIQUE.	Las consultas de lectura filtran activo = TRUE, y el UNIQUE (lectura_id, tipo) cuenta las alertas borradas: repetir el POST responde 409 (5.3).	Con 8 sensores el volumen es de unas 70 000 lecturas al año; para el 409 hay opciones baratas (R-05).
Demostrar la arquitectura de punta a punta y con seguridad (Sección 5).	Adecuación funcional (AC-01): alcance del producto.	El webhook, la evaluación de umbral y tendencia, los canales de correo y Telegram, la configuración de umbrales y la interfaz React están diseñados y no implementados (1.1, 3.5).	Lo evaluable en esta entrega es la arquitectura y su recorrido por capas. Lo pendiente está declarado y es lo primero que se construiría (6.3).

Atributo priorizado	Atributo sacrificado	Evidencia del compromiso	¿Por qué es aceptable en este proyecto?
Integridad de los datos en la base (CHECK, UNIQUE, FK) con un esquema simple de humedad.	Flexibilidad para otros tipos de sensor.	lectura.humedad tiene CHECK entre 0 y 100 y sensor no tiene tipo ni unidad (5.2); solo hay humedad (RT-04).	El caso solo trae sensores de humedad y otras variables están fuera de alcance (1.1). Si eso cambia, se agrega una migración (R-01).

6.2 Puntos de sensibilidad y riesgos técnicos

Un punto de sensibilidad es un elemento de la arquitectura cuyo cambio afecta directamente un atributo de calidad. Estos son los cinco que se identificaron, con su riesgo y su mitigación.

#	Punto de sensibilidad	Atributo afectado	Riesgo técnico	Probabilidad / Impacto	Mitigación propuesta
R-01	Esquema solo de humedad: lectura.humedad con CHECK entre 0 y 100 y sensor sin tipo ni unidad (5.2).	Mantenibilidad (AC-07) y adecuación funcional (AC-01)	Si entra un sensor de otra magnitud (lluvia en mm), el CHECK rechaza valores como 120; y si se aceptaran, se compararían con umbrales de humedad y darían alertas falsas.	Baja / Alto	Migración V2__tipo_sensor.sql: catálogo tipo_sensor (nombre y unidad), sensor.tipo_sensor_id y la columna humedad renombrada a valor; el rango se valida en la capa de negocio según el tipo. Se revisa si el cliente confirma otro sensor.
R-02	El celular receptor como único punto de entrada (ADR-001, RT-03).	Confiabilidad (AC-05)	Si el celular se apaga, pierde internet o falla la aplicación de reenvío, no llegan lecturas y nadie lo nota: vuelve el retraso que motivó el proyecto.	Media / Alto	Implementar HU-03: marcar un sensor en silencio si no reporta en más de dos intervalos (2 horas) y alertar. Tener un segundo celular con la misma aplicación que llame al mismo webhook. Confirmar los supuestos con el cliente.
R-03	Servidor único: la API y PostgreSQL comparten equipo (3.5).	Confiabilidad (AC-05)	Una falla del equipo detiene la API y la base a la vez, y sin guardia nocturna (RA-03) nadie la corrige hasta la mañana.	Media / Alto	Reinicio automático: la base ya lo tiene (restart: unless-stopped en docker-compose.yml) y falta la API (servicio del sistema operativo). Copias de seguridad programadas del volumen alerta_pgdata. Monitoreo externo gratuito de /actuator/health, que es público.

#	Punto de sensibilidad	Atributo afectado	Riesgo técnico	Probabilidad / Impacto	Mitigación propuesta
R-04	Secretos compartidos (JWT_SECRET, WEBHOOK_SECRET) y transporte sin TLS (ADR-002).	Seguridad (AC-06)	Quien obtenga JWT_SECRET puede firmar tokens de ADMINISTRADOR. Sin HTTPS, el token y el secreto del webhook viajan en claro. Rotar un secreto invalida los tokens o exige actualizar la aplicación del celular.	Media / Crítico	Secretos solo en variables de entorno (.env fuera de Git), de mínimo 32 caracteres; los valores de ejemplo de .env.example no se usan en producción. Rotación: cambiar la variable y reiniciar (los tokens vencen a los 60 minutos). Terminador TLS delante del puerto 8080 antes de producción.
R-05	Borrado lógico con UNIQUE (lectura_id, tipo) que cuenta las filas inactivas (5.2, ADR-003).	Adecuación funcional (AC-01) y mantenibilidad	Tras borrar una alerta no se puede crear otra del mismo tipo para esa lectura (409), así que un error al crearla no se corrige borrando y volviendo a crear. Las filas inactivas se acumulan.	Alta / Bajo	Corregir con PUT (nivel y mensaje) en lugar de borrar. Si el negocio lo exige, una migración que cambie el UNIQUE por un índice único parcial WHERE activo. Archivar las filas inactivas antiguas.

6.3 Conclusión arquitectónica

La arquitectura satisface los escenarios de la sección 1.3 solo en parte. El corte vertical demuestra que el patrón N-Capas funciona de punta a punta: una petición atraviesa controlador, servicio y repositorio hasta una fila en PostgreSQL, con errores centralizados, borrado lógico y acceso por rol con JWT, y cada regla vive en una sola capa, que es la base de la Mantenibilidad (AC-07). Todavía no hay evidencia de la Eficiencia (AC-02, ≤ 5 s) ni de la Confiabilidad (AC-05, dos canales), porque el webhook, la evaluación de umbral y tendencia y las notificaciones están diseñados pero no implementados. Si el proyecto pasara a producción, lo primero sería construir ese camino y medir el AC-02 en los logs; después, la detección de sensores en silencio (R-02) y el TLS delante de la API (R-04), que son los riesgos de mayor impacto de la tabla 6.2.